PROGRAMAÇÃO AVANÇADA EM PYTHON

Durante muitas lições passadas, cobrimos muitos tópicos relacionados ao Python! Nesta lição, focaremos em muitos dos itens "et cetera" não discutidos anteriormente. "Et cetera" significa literalmente "e o resto"! De fato, se você olhar a documentação do Python, encontrará muitas outras funcionalidades.

CONJUNTO

Em matemática, um conjunto seria considerado um conjunto de números sem duplicatas. Na janela do editor de texto, codifique da seguinte forma:

```
students = [
  {"name": "Hermione", "house": "Gryffindor"},
  {"name": "Harry", "house": "Gryffindor"},
  {"name": "Ron", "house": "Gryffindor"},
  {"name": "Draco", "house": "Slytherin"},
  {"name": "Padma", "house": "Ravenclaw"},
]

houses = []
for student in students:
  if student["house"] not in houses:
    houses.append(student["house"])

for house in sorted(houses):
  print(house)
```

Observe como temos uma lista de dicionários, cada um representando um aluno. Uma lista vazia chamada houses é criada. Iteramos por cada aluno em students. Se a casa de um aluno não estiver em houses, adicionamos à nossa lista de casas. Descobre-se que podemos usar as funcionalidades embutidas de conjunto para eliminar duplicatas.

Tradução livre do material ofertado no curso CS50's Introduction to Programming with Python

Na janela do editor de texto, codifique da seguinte forma:

```
students = [
  {"name": "Hermione", "house": "Gryffindor"},
  {"name": "Harry", "house": "Gryffindor"},
  {"name": "Ron", "house": "Gryffindor"},
  {"name": "Draco", "house": "Slytherin"},
  {"name": "Padma", "house": "Ravenclaw"},
]

houses = set()
for student in students:
  houses.add(student["house"])

for house in sorted(houses):
  print(house)
```

Observe como não é necessário incluir verificação para garantir que não haja duplicatas. O objeto set cuida disso automaticamente para nós. Para mais informações, consulte a documentação do Python sobre conjuntos.

VARIÁVEIS GLOBAIS

Em outras linguagens de programação, existe a noção de variáveis globais que são acessíveis a qualquer função.
Podemos aproveitar essa capacidade no Python. Na janela do editor de texto, codifique da seguinte forma:

```
balance = 0
```

```
def main():
    print("Balance:", balance)


if __name__ == "__main__":
    main()
```

Observe como criamos uma variável global chamada balance, fora de qualquer função. Como nenhum erro é apresentado ao executar o código acima, você pensaria que está tudo bem. No entanto, não está! Na janela do editor de texto, codifique da seguinte forma:

```
balance = 0


def main():
    print("Balance:", balance)
    deposit(100)
    withdraw(50)
    print("Balance:", balance)


def deposit(n):
    balance += n


def withdraw(n):
    balance -= n


if __name__ == "__main__":
```

```
main()
```

Observe como agora adicionamos a funcionalidade de adicionar e retirar fundos do saldo. No entanto, ao executar este código, nos deparamos com um erro! Vemos um erro chamado UnboundLocalError. Você pode supor que, pelo menos da maneira como codificamos balance e nossas funções deposit e withdraw, não podemos atribuir um valor a ele dentro de uma função.Para interagir com uma variável global dentro de uma função, a solução é usar a palavra-chave global. Na janela do editor de texto, codifique da seguinte forma:

```
balance = 0


def main():
    print("Balance:", balance)
    deposit(100)
    withdraw(50)
    print("Balance:", balance)


def deposit(n):
    global balance
    balance += n


def withdraw(n):
    global balance
    balance -= n


if __name__ == "__main__":
    main()
```

Observe como a palavra-chave global informa a cada função que balance não se refere a uma variável local: em vez disso, refere-se à variável global que originalmente colocamos no topo do nosso código. Agora, nosso código funciona! Utilizando nossos conhecimentos de programação orientada a objetos, podemos modificar nosso código para usar uma classe em vez de uma variável global. Na janela do editor de texto, codifique da seguinte forma:

```
class Account:
  def __init__(self):
    self._balance = 0

  @property
  def balance(self):
    return self._balance

  def deposit(self, n):
    self._balance += n

  def withdraw(self, n):
    self._balance -= n


def main():
  account = Account()
  print("Balance:", account.balance)
  account.deposit(100)
  account.withdraw(50)
  print("Balance:", account.balance)
```

```
if __name__ == "__main__":
  main()
```

Observe como usamos conta = Conta() para criar uma conta. As classes nos permitem resolver esse problema de precisar de uma variável global de forma mais limpa, pois essas variáveis de instância são acessíveis a todos os métodos dessa classe utilizando self. Em geral, variáveis globais devem ser usadas com muita parcimônia, se é que devem ser usadas!

CONSTANTES

Algumas linguagens permitem criar variáveis que são imutáveis, chamadas "constantes". As constantes permitem programar defensivamente e reduzir as oportunidades para valores importantes serem alterados. Na janela do editor de texto, codifique da seguinte forma:

```
MEOWS = 3

for _ in range(MEOWS):
  print("meow")
```

Observe que MEOWS é nossa constante neste caso. Constantes geralmente são indicadas por nomes de variáveis em maiúsculas e são colocadas no início do nosso código. Embora pareça uma constante, na realidade, o Python não possui um mecanismo para impedir que alteremos esse valor dentro do nosso código! Em vez disso, é uma questão de honra: se um nome de variável estiver escrito em maiúsculas, simplesmente não o altere!

Você pode criar uma classe "constant", agora entre aspas porque sabemos que o Python não suporta realmente "constants". Na janela do editor de texto, codifique da seguinte forma:

```
class Cat:
  MEOWS = 3

  def meow(self):
    for _ in range(Cat.MEOWS):
      print("meow")
```

```
cat = Cat()
cat.meow()
```

Como MEOWS é definido fora de qualquer método específico da classe, todos têm acesso a esse valor através de Cat.MEOWS.

SUPER DICAS

Em outras linguagens de programação, você expressa explicitamente qual tipo de variável deseja usar. Como vimos anteriormente no curso, o Python não exige a declaração explícita de tipos. No entanto, é uma boa prática garantir que todas as suas variáveis sejam do tipo correto. mypy é um programa que pode ajudá-lo a testar se todas as suas variáveis são do tipo correto. Você pode instalar o mypy executando no seu terminal: pip install mypy. Na janela do editor de texto, codifique da seguinte forma:

```
def meow(n):
  for _ in range(n):
    print("meow")


number = input("Number: ")
meow(number)
```

Você já pode ver que número = input("Número: ") retorna uma string, não um int. Mas meow provavelmente desejará um int!

Você pode adicionar uma dica de tipo para dar ao Python uma dica sobre que tipo de variável mia deve esperar. Na janela do editor de texto, codifique da seguinte forma:

```
def meow(n: int):
  for _ in range(n):
```

```
    print("meow")


number = input("Number: ")
meow(number)
```

Observe que nosso programa ainda lança um erro.

Depois de instalar o mypy, execute mypy meows.py no terminal. O mypy fornecerá orientações sobre como corrigir esse erro. Você pode anotar todas as suas variáveis. Na janela do editor de texto, codifique da seguinte forma:

```
def meow(n: int):
  for _ in range(n):
    print("meow")


number: int = input("Number: ")
meow(number)
```

Observe como o número agora recebe uma dica de tipo.

Novamente, executar mypy meows.py no terminal fornece feedback muito mais específico para você, o programador. Podemos corrigir nosso último erro codificando da seguinte forma:

```
def meow(n: int):
  for _ in range(n):
    print("meow")


number: int = int(input("Number: "))
```

```
meow(number)
```

Observe como executar mypy agora não produz erros porque convertemos nossa entrada para um número inteiro.

Vamos introduzir um novo erro assumindo que mia nos retornará uma string, ou str. Na janela do editor de texto, codifique da seguinte forma:

```
def meow(n: int):
  for _ in range(n):
    print("meow")


number: int = int(input("Number: "))
meows: str = meow(number)
print(meows)
```

Observe como a função mia tem apenas um efeito colateral. Como só tentamos imprimir "meow", não retornar um valor, é lançado um erro quando tentamos armazenar o valor de retorno de meow em meows.

Podemos usar ainda mais dicas de tipo para verificar erros, desta vez anotando os valores de retorno das funções. Na janela do editor de texto, codifique da seguinte forma:

```
def meow(n: int) -> None:
  for _ in range(n):
    print("meow")


number: int = int(input("Number: "))
meows: str = meow(number)
print(meows)
```

Observe como a notação -> None informa ao mypy que não há valor de retorno.

Podemos modificar nosso código para retornar uma string se desejarmos:

```
def meow(n: int) -> str:
    return "meow\n" * n


number: int = int(input("Number: "))
meows: str = meow(number)
print(meows, end="")
```

Observe como armazenamos em miaus múltiplas strs. Executar mypy não produz erros.

DOCSTRINGS

Uma maneira padrão de comentar o propósito de sua função é usar uma docstring. Na janela do editor de texto, codifique da seguinte forma:

```
def meow(n):
    """Meow n times."""
    return "meow\n" * n


number = int(input("Number: "))
meows = meow(number)
print(meows, end="")
```

Observe como as três aspas duplas designam o que a função faz.

Você pode usar docstrings para padronizar como documenta as características de uma função. Na janela do editor de texto, codifique da seguinte forma:

```
def meow(n):
    """
    Meow n times.

    :param n: Number of times to meow
    :type n: int
    :raise TypeError: If n is not an int
    :return: A string of n meows, one per line
    :rtype: str
    """
    return "meow\n" * n


number = int(input("Number: "))
meows = meow(number)
print(meows, end="")
```

Observe como vários argumentos docstring estão incluídos. Por exemplo, descreve os parâmetros obtidos pela função e o que é retornado pela função.

Ferramentas estabelecidas, como o Sphinx, podem ser usadas para analisar documentos e criar automaticamente documentação para nós na forma de páginas da web e arquivos PDF para que você possa publicar e compartilhar com outras pessoas.

ARGPARSE

Suponha que queiramos usar argumentos de linha de comando em nosso programa. Na janela do editor de texto, codifique da seguinte forma:

```
import sys
```

```
if len(sys.argv) == 1:
  print("meow")
elif len(sys.argv) == 3 and sys.argv[1] == "-n":
  n = int(sys.argv[2])
  for _ in range(n):
    print("meow")
else:
  print("usage: meows.py [-n NUMBER]")
```

Observe como sys é importado, de onde obtemos acesso a sys.argv — uma matriz de argumentos de linha de comando fornecidos ao nosso programa quando executado. Podemos usar várias instruções if para verificar se o uso executou nosso programa corretamente.

Vamos supor que este programa ficará muito mais complicado. Como poderíamos verificar todos os argumentos que poderiam ser inseridos pelo usuário? Poderemos desistir se tivermos mais do que alguns argumentos de linha de comando!

Felizmente, argparse é uma biblioteca que lida com toda a análise de strings complicadas de argumentos de linha de comando. Na janela do editor de texto, codifique da seguinte forma:

```
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("-n")
args = parser.parse_args()

for _ in range(int(args.n)):
  print("meow")
```

Observe como argparse é importado em vez de sys. Um objeto chamado analisador é criado a partir de uma classe ArgumentParser. O método add_argument dessa classe é usado para informar ao argparse quais argumentos devemos esperar do usuário quando ele executa nosso programa. Finalmente, executar o método parse_args do analisador garante que todos os argumentos foram incluídos corretamente pelo usuário.

Também podemos programar de forma mais limpa, de forma que nosso usuário possa obter algumas informações sobre o uso adequado de nosso código quando não conseguir usar o programa corretamente. Na janela do editor de texto, codifique da seguinte forma:

```
import argparse

parser = argparse.ArgumentParser(description="Meow like a cat")
parser.add_argument("-n", help="number of times to meow")
args = parser.parse_args()

for _ in range(int(args.n)):
  print("meow")
```

Observe como o usuário recebe alguma documentação. Especificamente, é fornecido um argumento de ajuda. Agora, se o usuário executar python meows.py --help ou -h, o usuário receberá algumas dicas sobre como usar este programa.

Podemos melhorar ainda mais este programa. Na janela do editor de texto, codifique da seguinte forma:

```
import argparse

parser = argparse.ArgumentParser(description="Meow like a cat")
parser.add_argument("-n", default=1, help="number of times to meow", type=int)
args = parser.parse_args()

for _ in range(args.n):
  print("meow")
```

Observe como não apenas a documentação de ajuda está incluída, mas você também pode fornecer um valor padrão quando nenhum argumento for fornecido pelo usuário.

DESEMPACOTANDO

Não seria bom poder dividir uma única variável em duas variáveis? Na janela do editor de texto, codifique da seguinte forma:

```
first, _ = input("What's your name? ").split(" ")
print(f"hello, {first}")
```

Observe como este programa tenta obter o primeiro nome de um usuário dividindo-o ingenuamente em um único espaço.

Acontece que existem outras maneiras de descompactar variáveis. Você pode escrever um código mais poderoso e elegante entendendo como descompactar variáveis de maneiras aparentemente mais avançadas. Na janela do editor de texto, codifique da seguinte forma:

```
def total(galleons, sickles, knuts):
  return (galleons * 17 + sickles) * 29 + knuts

print(total(100, 50, 25), "Knuts")
```

Observe como isso retorna o valor total de Knuts.

E se quiséssemos armazenar nossas moedas em uma lista? Na janela do editor de texto, codifique da seguinte forma:

```
def total(galleons, sickles, knuts):
  return (galleons * 17 + sickles) * 29 + knuts
```

```
coins = [100, 50, 25]

print(total(coins[0], coins[1], coins[2]), "Knuts")
```

Observe como uma lista chamada moedas é criada. Podemos passar cada valor indexando usando 0, 1 e assim por diante.

Isso está ficando bastante detalhado. Não seria bom se pudéssemos simplesmente passar a lista de moedas para a nossa função?

Para habilitar a possibilidade de passar a lista inteira, podemos usar o unpacking. Na janela do editor de texto, codifique da seguinte forma:

```
def total(galleons, sickles, knuts):
  return (galleons * 17 + sickles) * 29 + knuts


coins = [100, 50, 25]

print(total(*coins), "Knuts")
```

Observe como um * descompacta a sequência da lista de moedas e passa cada um de seus elementos individuais para o total.

Suponha que pudéssemos passar os nomes da moeda em qualquer ordem? Na janela do editor de texto, codifique da seguinte forma:

```
def total(galleons, sickles, knuts):
  return (galleons * 17 + sickles) * 29 + knuts


print(total(galleons=100, sickles=50, knuts=25), "Knuts")
```

Observe como isso ainda é calculado corretamente.

Quando você começar a falar sobre “nomes” e “valores”, os dicionários podem começar a vir à mente! Você pode implementar isso como um dicionário. Na janela do editor de texto, codifique da seguinte forma:

```
def total(galleons, sickles, knuts):
  return (galleons * 17 + sickles) * 29 + knuts


coins = {"galleons": 100, "sickles": 50, "knuts": 25}

print(total(coins["galleons"], coins["sickles"], coins["knuts"]), "Knuts")
```

Observe como um dicionário chamado moedas é fornecido. Podemos indexá-lo usando chaves, como “galeões” ou “foices”.

Como a função total espera três argumentos, não podemos passar um dicionário. Podemos usar a descompactação para ajudar com isso. Na janela do editor de texto, codifique da seguinte forma:

```
def total(galleons, sickles, knuts):
  return (galleons * 17 + sickles) * 29 + knuts


coins = {"galleons": 100, "sickles": 50, "knuts": 25}

print(total(**coins), "Knuts")
```

Observe como ** permite descompactar um dicionário. Ao descompactar um dicionário, ele fornece as chaves e os valores.

ARGS E KWARGS

Lembre-se da documentação impressa que vimos anteriormente neste curso:

```
print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)
```

Args são argumentos posicionais, como aqueles que fornecemos para imprimir como print("Hello", "World").

kwargs são argumentos nomeados, ou “argumentos de palavras-chave”, como aqueles que fornecemos para impressão como print(end="").

Como vemos no protótipo da função print acima, podemos dizer à nossa função para esperar um número de argumentos posicionais atualmente desconhecidos. Também podemos dizer para esperar um número atualmente desconhecido de argumentos de palavras-chave. Na janela do editor de texto, codifique da seguinte forma:

```
def f(*args, **kwargs):
  print("Positional:", args)


f(100, 50, 25)
```

Observe como a execução deste código será impressa como argumentos posicionais.

Podemos até passar argumentos nomeados. Na janela do editor de texto, codifique da seguinte forma:

```
def f(*args, **kwargs):
  print("Named:", kwargs)


f(galleons=100, sickles=50, knuts=25)
```

Observe como os valores nomeados são fornecidos na forma de um dicionário.

Pensando na função print acima, você pode ver como *objects aceita qualquer número de argumentos posicionais.

MAPA

No início, começamos com a programação processual.

Mais tarde, revelamos que Python é uma linguagem de programação orientada a objetos.

Vimos dicas de programação funcional, onde funções têm efeitos colaterais sem valor de retorno. Podemos ilustrar isso na janela do editor de texto, digite o código grito.py e codifique da seguinte forma:

```
def main():
  yell("This is CS50")


def yell(word):
  print(word.upper())


if __name__ == "__main__":
  main()
```

Observe como a função de grito é simplesmente gritada.

Não seria bom gritar uma lista de palavras ilimitadas? Modifique seu código da seguinte maneira:

```
def main():
  yell(["This", "is", "CS50"])
```

```
def yell(words):
    uppercased = []
    for word in words:
        uppercased.append(word.upper())
    print(*uppercased)


if __name__ == "__main__":
    main()
```

Observe que acumulamos as palavras maiúsculas, iterando cada uma das palavras e colocando-as em maiúsculas. A lista em maiúsculas é impressa utilizando * para descompactá-la.

Removendo os colchetes, podemos passar as palavras como argumentos. Na janela do editor de texto, codifique da seguinte forma:

```
def main():
    yell("This", "is", "CS50")


def yell(*words):
    uppercased = []
    for word in words:
        uppercased.append(word.upper())
    print(*uppercased)


if __name__ == "__main__":
    main()
```

Observe como *words permite que muitos argumentos sejam usados pela função.

Map permite mapear uma função para uma sequência de valores. Na prática, podemos codificar da seguinte forma:

```
def main():
  yell("This", "is", "CS50")


def yell(*words):
  uppercased = map(str.upper, words)
  print(*uppercased)


if __name__ == "__main__":
  main()
```

Observe como map leva dois argumentos. Primeiro, é necessária uma função que queremos aplicar a cada elemento de uma lista. Em segundo lugar, é necessária a própria lista, à qual aplicaremos a função acima mencionada. Conseqüentemente, todas as palavras em palavras serão entregues à função str.upper e retornadas em maiúsculas.

COMPREENSÕES DE LISTA

As compreensões de lista permitem que você crie uma lista dinamicamente em uma linha elegante.

Podemos implementar isso em nosso código da seguinte maneira:

```
def main():
```

```
    yell("This", "is", "CS50")


def yell(*words):
    uppercased = [arg.upper() for arg in words]
    print(*uppercased)


if __name__ == "__main__":
    main()
```

Observe como, em vez de usar map, escrevemos uma expressão Python entre colchetes. Para cada argumento, .upper é aplicado a ele.

Levando esse conceito adiante, vamos mudar para outro programa.

Na janela do editor de texto, digite o código gryffindors.py e codifique da seguinte forma:

```
students = [
    {"name": "Hermione", "house": "Gryffindor"},
    {"name": "Harry", "house": "Gryffindor"},
    {"name": "Ron", "house": "Gryffindor"},
    {"name": "Draco", "house": "Slytherin"},
]

gryffindors = []
for student in students:
    if student["house"] == "Gryffindor":
        gryffindors.append(student["name"])

for gryffindor in sorted(gryffindors):
```

```
  print(gryffindor)
```

Observe que temos uma condicional enquanto criamos nossa lista. Se a casa do aluno for a Grifinória, acrescentamos o aluno à lista de nomes. Finalmente, imprimimos todos os nomes.

De forma mais elegante, podemos simplificar este código com uma compreensão de lista da seguinte forma:

```
students = [
  {"name": "Hermione", "house": "Gryffindor"},
  {"name": "Harry", "house": "Gryffindor"},
  {"name": "Ron", "house": "Gryffindor"},
  {"name": "Draco", "house": "Slytherin"},
]

gryffindors = [
  student["name"] for student in students if student["house"] == "Gryffindor"
]

for gryffindor in sorted(gryffindors):
  print(gryffindor)
```

Observe como a compreensão da lista está em uma única linha!

FILTRO

Usar a função de filtro(filter) do Python nos permite retornar um subconjunto de uma sequência para a qual uma determinada condição é verdadeira.

Na janela do editor de texto, codifique da seguinte forma:

```
students = [
  {"name": "Hermione", "house": "Gryffindor"},
  {"name": "Harry", "house": "Gryffindor"},
  {"name": "Ron", "house": "Gryffindor"},
  {"name": "Draco", "house": "Slytherin"},
]


def is_gryffindor(s):
  return s["house"] == "Gryffindor"


gryffindors = filter(is_gryffindor, students)

for gryffindor in sorted(gryffindors, key=lambda s: s["name"]):
  print(gryffindor["name"])
```

Observe como uma função chamada is_gryffindor é criada. Esta é a nossa função de filtragem que pegará a casa do aluno e retornará Verdadeiro ou Falso dependendo se a casa do aluno é a Grifinória. Você pode ver que a nova função de filtro leva dois argumentos. Primeiro, é necessária a função que será aplicada a cada elemento em uma sequência – neste caso, is_gryffindor. Em segundo lugar, é necessária a sequência à qual aplicará a função de filtragem – neste caso, alunos. Na Grifinória, devemos ver apenas os alunos que estão na Grifinória.

Filter também pode usar funções lambda da seguinte maneira:

```
students = [
  {"name": "Hermione", "house": "Gryffindor"},
  {"name": "Harry", "house": "Gryffindor"},
  {"name": "Ron", "house": "Gryffindor"},
  {"name": "Draco", "house": "Slytherin"},
```

```
]


gryffindors = filter(lambda s: s["house"] == "Gryffindor", students)


for gryffindor in sorted(gryffindors, key=lambda s: s["name"]):
    print(gryffindor["name"])
```

Observe como a mesma lista de alunos é fornecida.

COMPREENSÕES DE DICIONÁRIO

Podemos aplicar a mesma ideia por trás da compreensão de listas aos dicionários. Na janela do editor de texto, codifique da seguinte forma:

```
students = ["Hermione", "Harry", "Ron"]


gryffindors = []


for student in students:
    gryffindors.append({"name": student, "house": "Gryffindor"})


print(gryffindors)
```

Observe como este código (ainda!) não usa nenhuma compreensão. Em vez disso, segue os mesmos paradigmas que vimos antes.

Agora podemos aplicar compreensões de dicionário modificando nosso código da seguinte maneira:

```
students = ["Hermione", "Harry", "Ron"]
```

```
gryffindors = [{"name": student, "house": "Gryffindor"} for student in students]

print(gryffindors)
```

Observe como todo o código anterior é simplificado em uma única linha onde a estrutura do dicionário é fornecida para cada aluno em alunos.

Podemos ainda simplificar ainda mais da seguinte forma:

```
students = ["Hermione", "Harry", "Ron"]

gryffindors = {student: "Gryffindor" for student in students}

print(gryffindors)
```

Observe como o dicionário será construído com pares de valores-chave.

ENUMERAR

Podemos desejar fornecer alguma classificação de cada aluno. Na janela do editor de texto, codifique da seguinte forma:

```
students = ["Hermione", "Harry", "Ron"]

for i in range(len(students)):
  print(i + 1, students[i])
```

Observe como cada aluno é enumerado ao executar este código.

Utilizando a enumeração, podemos fazer o mesmo:

```
students = ["Hermione", "Harry", "Ron"]

for i, student in enumerate(students):
  print(i + 1, student)
```

Observe como enumerate apresenta o índice e o valor de cada aluno.

GERADORES E ITERADORES

Em Python, existe uma maneira de proteger contra a falta de recursos do sistema, pois os problemas que eles estão abordando se tornam muito grandes.

Nos Estados Unidos, é costume “contar ovelhas” mentalmente quando alguém está com dificuldade para adormecer.

Na janela do editor de texto, digite o código sleep.py e codifique da seguinte forma:

```
n = int(input("What's n? "))
for i in range(n):
  print("🐑" * i)
```

Observe como este programa contará o número de ovelhas que você solicitar.

Podemos tornar nosso programa mais sofisticado adicionando uma função principal codificando da seguinte forma:

```
def main():
```

```
    n = int(input("What's n? "))
    for i in range(n):
        print(" " * i)


if __name__ == "__main__":
    main()
```

Observe como uma função principal é fornecida.

Adquirimos o hábito de abstrair partes do nosso código.

Podemos chamar uma função ovelha modificando nosso código da seguinte maneira:

```
def main():
    n = int(input("What's n? "))
    for i in range(n):
        print(sheep(i))


def sheep(n):
    return " " * n


if __name__ == "__main__":
    main()
```

Observe como a função principal faz a iteração.

Podemos fornecer às ovelhas mais habilidades. Na janela do editor de texto, codifique da seguinte forma:

```
def main():
  n = int(input("What's n? "))
  for s in sheep(n):
    print(s)


def sheep(n):
  flock = []
  for i in range(n):
    flock.append("🐑" * i)
  return flock


if __name__ == "__main__":
  main()
```

Observe como criamos um rebanho de ovelhas e devolvemos o rebanho.

Ao executar nosso código, você pode tentar diferentes números de ovelhas, como 10, 1.000 e 10.000. E se você pedisse 1.000.000 de ovelhas, seu programa poderia travar completamente ou travar. Como você tentou gerar uma lista enorme de ovelhas, seu computador pode estar tendo dificuldades para concluir o cálculo.

O gerador de rendimento pode resolver esse problema retornando uma pequena parte dos resultados de cada vez. Na janela do editor de texto, codifique da seguinte forma:

```
def main():
    n = int(input("What's n? "))
    for s in sheep(n):
        print(s)


def sheep(n):
    for i in range(n):
        yield " " * i


if __name__ == "__main__":
    main()
```

Observe como o “yield” fornece apenas um valor por vez enquanto o loop for continuar funcionando.

PARABENS!!!

Ao sair deste curso, você terá mais um modelo mental e uma caixa de ferramentas para resolver problemas relacionados à programação.

Primeiro, você aprendeu sobre funções e variáveis.

Segundo, você aprendeu sobre condicionais.

Terceiro, você aprendeu sobre loops.

Quarto, você aprendeu sobre exceções.

Tradução livre do material ofertado no curso CS50's Introduction to Programming with Python

Quinto, você aprendeu sobre bibliotecas.

Sexto, você aprendeu sobre testes unitários.

Sétimo, você aprendeu sobre E/S de arquivos.

Oitavo, você aprendeu sobre expressões regulares.

Mais recentemente, você aprendeu sobre programação orientada a objetos.

Hoje, você aprendeu sobre muitas outras ferramentas que pode usar.

ESTE FOI O CS50!

Criando um programa final juntos, digite o código say.py na janela do terminal e codifique da seguinte forma:

```
import cowsay
import pyttsx3

engine = pyttsx3.init()
this = input("What's this? ")
cowsay.cow(this)
engine.say(this)
engine.runAndWait()
```

Observe como a execução deste programa proporciona uma despedida animada.

Nossa grande esperança é que você use o que aprendeu neste curso para resolver problemas reais do mundo, tornando nosso globo um lugar melhor.

ESTE FOI O CS50!